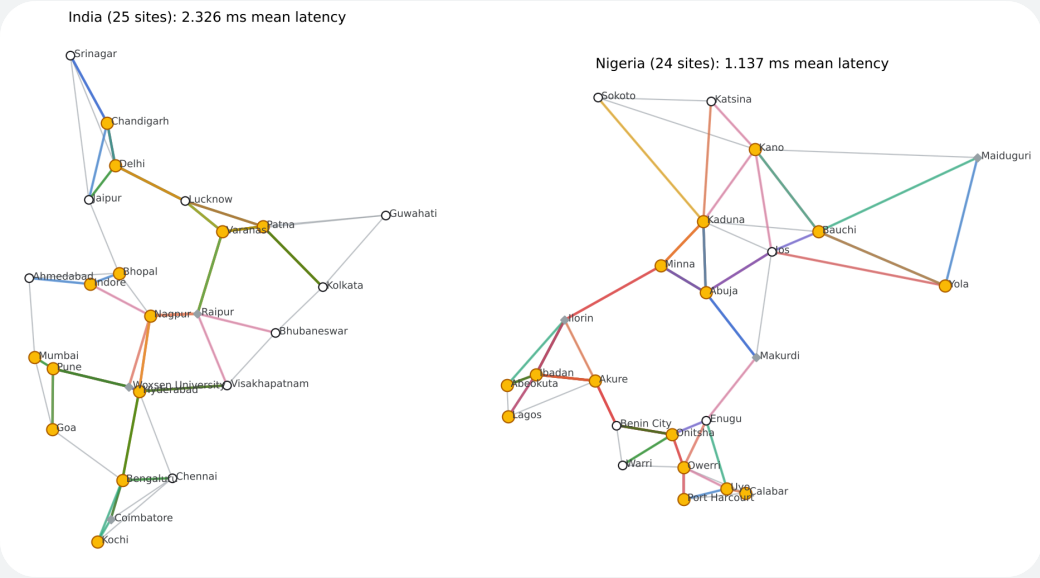
DESIGN AND ANALYSIS OF ALGORITHMS · PROJECT

Lightpath.

Latency-aware cloud task placement on network graphs

Where should copies of each cloud service run, so every user is one short path away, without overloading servers or breaking the budget?



Naman Singla · 25WU0101084
Mohak Gupta · 25WU0101074

Faculty: Dr. Monday Jubrin Abdullahi
Section: CSE RHINOS · 2026–27

THE PROBLEM

A cloud runs many services for users spread across many cities

If a user in Kochi is served from Delhi, every request crosses the country. Put a copy closer and it is fast — but every copy costs money and uses server capacity.

Our goal

- **Minimise average user latency** (ms)
- never exceed a server's **CPU / memory**
- stay within the **budget** ($ per hour)

Why it is hard: this contains the classic **p-median** problem, which is **NP-hard**. No known fast algorithm always finds the exact best answer, so we combine exact algorithms with smart greedy ones — and measure how close they get.

Real-life examples

- Netflix / YouTube caching videos in nearby cities
- Banks placing payment servers per region
- Woxsen campus: which building hosts the Wi-Fi login server

Three questions decide everything: **how far** are cities, **how many copies** can we afford, and **where** do they go?

HOW WE MODEL IT

Turn the cloud into a weighted graph

Nodes

Servers / cities

each has CPU, memory and a $/hour cost

Edges

Fibre links

weight = **latency in ms** (from real distance)

Demand

Users

requests per second from each city for each service

Storage

Adjacency list

O(V+E) memory; real networks are sparse

Average latency = **Σ** (requests from city *c*) × (shortest distance from *c* to its nearest copy) ÷ total requests

10 networks to test on

Real: India (25 cities incl. Woxsen), Nigeria (24 cities), Woxsen campus (23 buildings), Abilene & NSFNET (US research networks). **Synthetic:** Waxman, Erdős–Rényi, Barabási–Albert, data-centre fat-tree — so we can vary size and density.

Transfer cost with rebates

We also model data cost in **$/GB**. Some peering links pay a rebate → **negative edge weights**. We proved our cost model never creates a **negative cycle** (potential function, Lemma 1).

THE SOLUTION

Seven stages — each a classic algorithm chosen for what it guarantees



1	2	3	4	5	6	7
Latency matrix	**Backbone**	**Routing**	**Server activation**	**Replica sizing**	**Placement**	**Refinement**
Floyd–Warshall / Johnson	Prim / Kruskal MST	Dijkstra / Bellman-Ford	0/1 knapsack DP	Multiple-choice knapsack	4 greedy schedulers	Local search
How far is every city from every other?	Cheapest links connecting all cities	Which path does each request take?	Which servers to switch on within budget?	How many copies of each service?	Exactly where does each copy go?	Move / swap copies until nothing improves

Exact check (the referee): on smaller networks a **mixed-integer program** (MILP, HiGHS solver) finds the **proven optimum**. Every heuristic reports its **gap** from it — so we never just claim our answer is good, we prove how good.

Why four shortest-path algorithms?

One source

Dijkstra

Settles the closest city first; its distance is then final. **Fast**, stops early at the destination.

`O((V+E) log V)`

Fails with negative weights

One source

Bellman-Ford

Relaxes every edge V−1 rounds. **Handles rebates** (negative costs) and detects negative cycles.

`O(V·E)`

Used for $/GB transfer routes

All pairs

Floyd–Warshall

DP over intermediate cities: d[i][j] = min(d[i][j], d[i][k]+d[k][j]). Builds the full latency table.

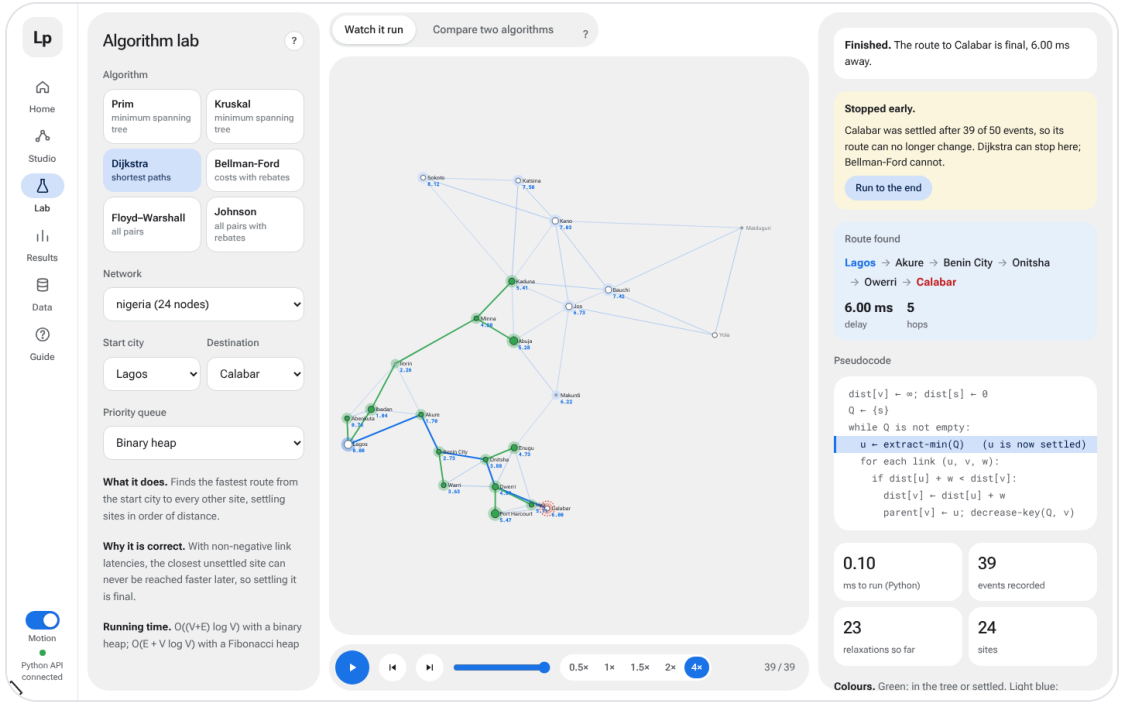
`Θ(V³)`

All pairs

Johnson

Bellman-Ford once to reweight edges ≥ 0, then Dijkstra from every city. Better on **sparse** graphs.

`O(VE + V² log V)`



Algorithm Lab: Dijkstra from Lagos to Calabar, step by step with pseudocode — the blue line is the final route. The code **picks automatically**: sparse → Johnson/Dijkstra, dense → Floyd–Warshall.

STAGES 2, 4 & 5 · SPANNING TREES AND DYNAMIC PROGRAMMING

Build the backbone, then spend the budget wisely

Minimum spanning tree

Prim vs Kruskal

Prim grows one tree using a priority queue. **Kruskal** sorts edges and uses **union-find** to skip cycles.

Both give the **same cheapest tree** (cut property). Rule: Prim if **E > V log V** (dense), else Kruskal.

`O(E log V) · O(E log E)`

Measured: sparse → Kruskal 1.5 ms vs Prim 3.7 ms; dense → Prim 42 ms vs Kruskal 126 ms

0/1 knapsack DP

Which servers to switch on?

Each server is **on or off** (0/1), has a cost and a benefit, and the budget is the knapsack capacity. DP gives the **exact** best set.

`θ(n·W), W = budget in $0.01 steps`

Measured: grows linearly, ≈ 97 ns per table cell

Multiple-choice knapsack

How many copies?

For each service pick **exactly one option** — 1, 2, 3... copies — each with a cost and a latency saving, all within the VM budget.

`θ(W · Σ options)`

Budget $2.41 → $7.51/h cut latency 4.34 → 1.28 ms (−70%)

STAGES 6 & 7 · THE NP-HARD PART

Greedy placement + local search, checked against the exact optimum

Four greedy schedulers

- **Demand-first:** copies go where most users are
- **Marginal gain:** add the copy that cuts latency the most (CELF lazy updates)
- **Regret:** first serve the service that loses most if it misses its best city
- **FFD packing:** first-fit-decreasing bin packing — fits capacity first

Local search

Try **moving** one copy or **swapping** two; keep any change that lowers latency; repeat until nothing improves.

Why not just solve it exactly? The MILP grows exponentially: 27 ms at 20 servers → 1.7 s at 160. Greedy + local search is **≈5× faster** and almost as good.

Distance from the proven optimum

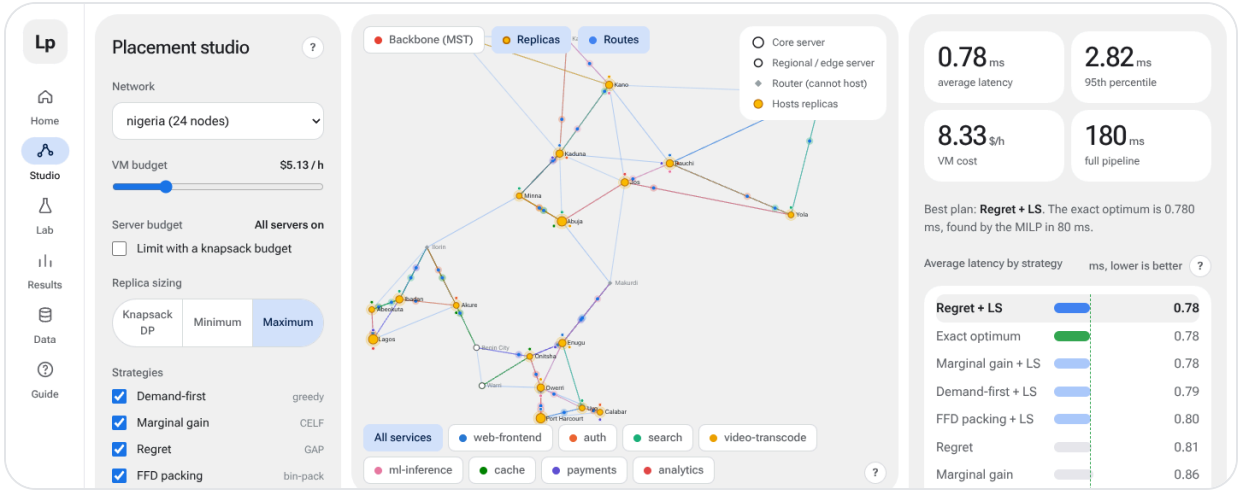
Method	Greedy alone	+ Local search
Random placement	167 %	—
FFD packing	38.4 %	**≈0.5 %**
Demand-first	16.7 %	**≈0.5 %**
Regret	9.2 %	**≈0.5 %**
Marginal gain	8.7 %	**≈0.5 %**

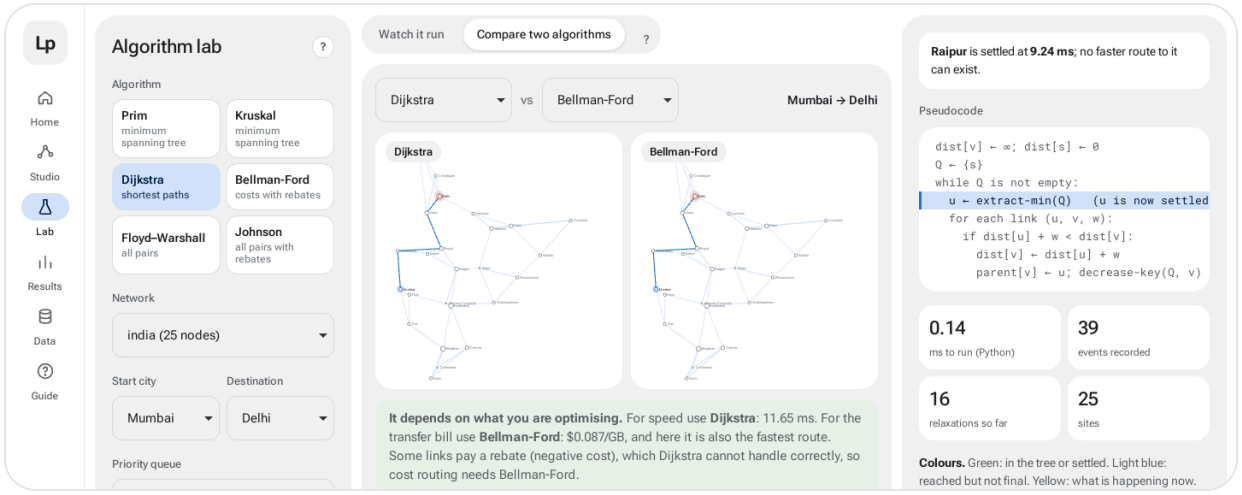
Averaged over Waxman networks of 20–240 servers. Worst case after local search: about **3 %**.

When servers are nearly full (E5), only **FFD packing** always found a valid plan — so the solver uses it as a fallback.

THE PLATFORM

Everything runs in an interactive website





Home

Live map of India or Nigeria — moving dots are user requests on their shortest path.

Placement Studio

Pick a network and budget → **Solve**. Map, every strategy vs the optimum, routes, server load and advice.

Algorithm Lab

Watch each algorithm from a start city to a destination at 0.5×–4×, or **compare two** and see which suits the trip.

Results · Data · Guide

8 experiment charts, all 10 networks to download, and a guided tour.

Python engine (NumPy, SciPy HiGHS) · FastAPI · React. Works offline too.

EXPERIMENTS · WHAT WE MEASURED

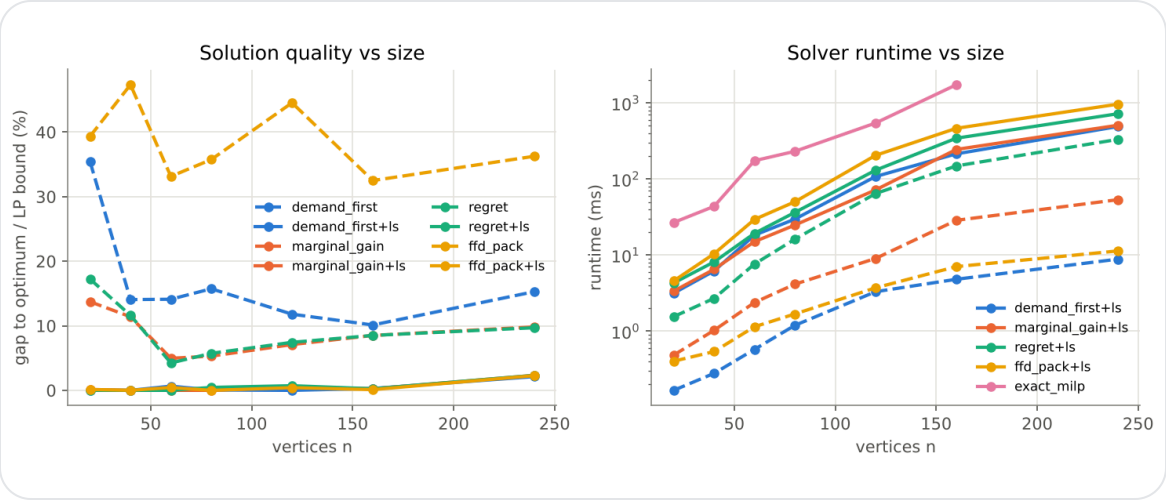
Eight experiments, one big lesson: measure, don't assume

2.33 ms
average latency, India network

−64%
vs a random placement

≈0.5%
from the proven optimum

191
automated tests, all passing



Fibonacci heap: best Big-O, but **slower** in practice (25.9 vs 17.7 ms) — pointer overhead beats the theory.

All pairs: NumPy Floyd–Warshall beat Johnson at n = 400 (277 vs 5015 ms dense).

MST: Kruskal wins sparse, Prim wins dense — exactly as theory predicts.

Budget & density: more money or more links → lower latency, with diminishing returns.

CONCLUSION

Right algorithm for each question → fast, near-optimal, provable plans

What we built

A complete system: 6 graph algorithms, 2 knapsack DPs, 4 greedy schedulers, 4 priority queues, an exact solver, 10 networks, 8 experiments and an interactive website.

What we found

Greedy + local search lands within **≈0.5 %** of the optimum, ≈5× faster than the exact solver. Placement cut latency by up to **64 %**.

What we learned

Big-O is a guide, not a verdict — constant factors and graph density decide the winner. Always check heuristics against an exact answer.

Future work: changing traffic over time, failures of links, and larger networks with faster approximate all-pairs methods.

Thank you.

Questions? Live demo on the website.

Naman Singla · **Mohak Gupta**
Guided by Dr. Monday Jubrin Abdullahi
CSE RHINOS · Woxsen University